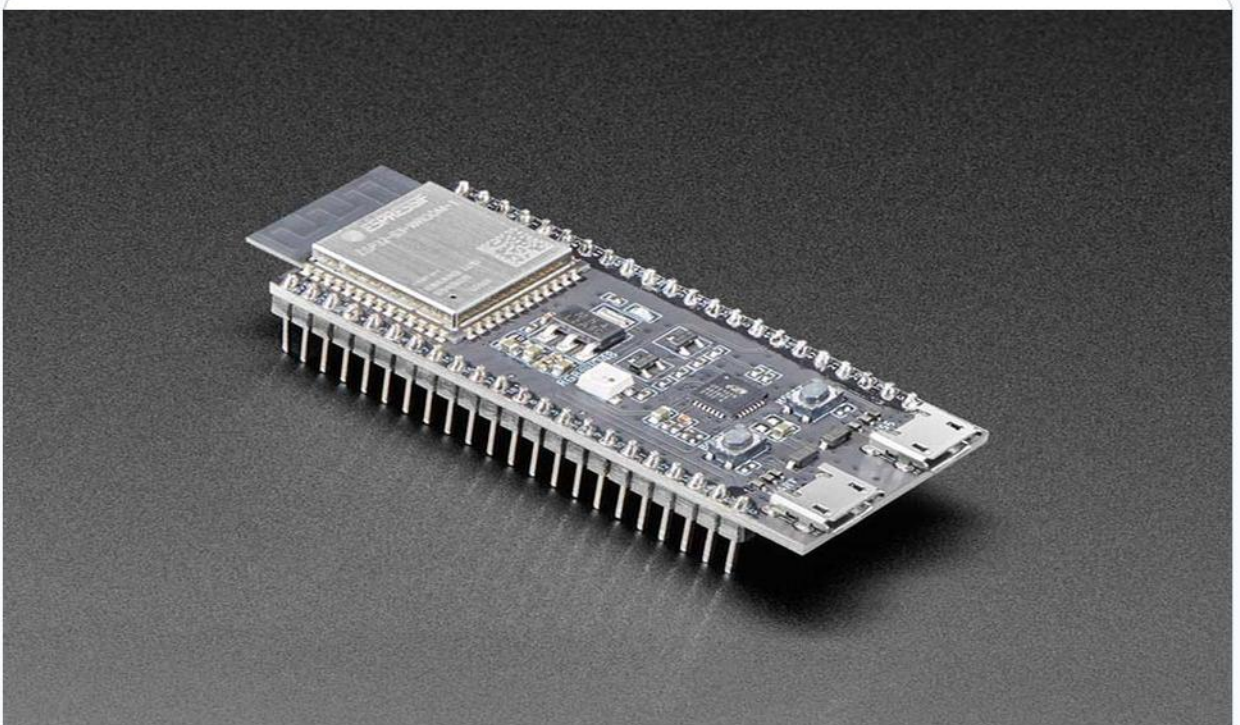


Running a 1.84M Generative Micro-LM on the ESP32-S3

100% Offline • Bare-Metal C++ • ~12-13 tokens/sec



ESP32-S3 DevKit | Xtensa 240MHz | 8MB PSRAM | INT8 (1.79 MB)

Brain for a Mini Desktop Robot

THE CURIOSITY

Can an ESP32 think locally?

Building an on-device language model for a desktop robot companion without cloud APIs, monthly co

THE PERSONA

Kibo (希望 - Hope)

Friendly conversational character with emotion tags ([HAPPY], [NEUTRAL]) to drive physical robot ges

THE FRAMEWORK

ESP32 Micro-LM Engine

A lightweight, modular, and fully customizable bare-metal C++ inference engine designed for \$5 micr

ESP32-S3 Hardware & Memory Map

COMPUTE ENGINE

240 MHz — Dual-Core Xtensa LX7

- Pure bare-metal C++ forward pass (Zero runtime bloat)
- SIMD-aligned vector dot-products
- Deterministic execution timing without garbage collection

MEMORY ALLOCATION

8 MB — Octal PSRAM (80MHz)

- 1.79 MB INT8 Model Weights directly mapped in PSRAM
- Dynamic 128-token Key-Value (KV) Cache
- Zero-wait working buffers for multi-head attention

MODEL ARCHITECTURE

1.84M — Causal Transformer Parameters

- 4 Transformer Layers | 192 Hidden Dimensions | 4 Heads
- 91 Vocabulary Tokens (Subwords + Control Tags)
- 100% Offline execution with zero cloud telemetry

From PyTorch to Bare-Metal C++

01

Dataset & Emotions

Curated conversational dialogue with [HAPPY] & [NEUTRAL] tags.

02

PyTorch Training

Trained 4-Layer Causal Transformer (1.84M params, 192 dim).

03

INT8 Quantization

Compressed model payload from 7.02 MB -> 1.79 MB (100% FP32 fidelity).

04

Bare-Metal C++ Engine

Custom forward pass and KV-cache with zero framework runtime overhead.

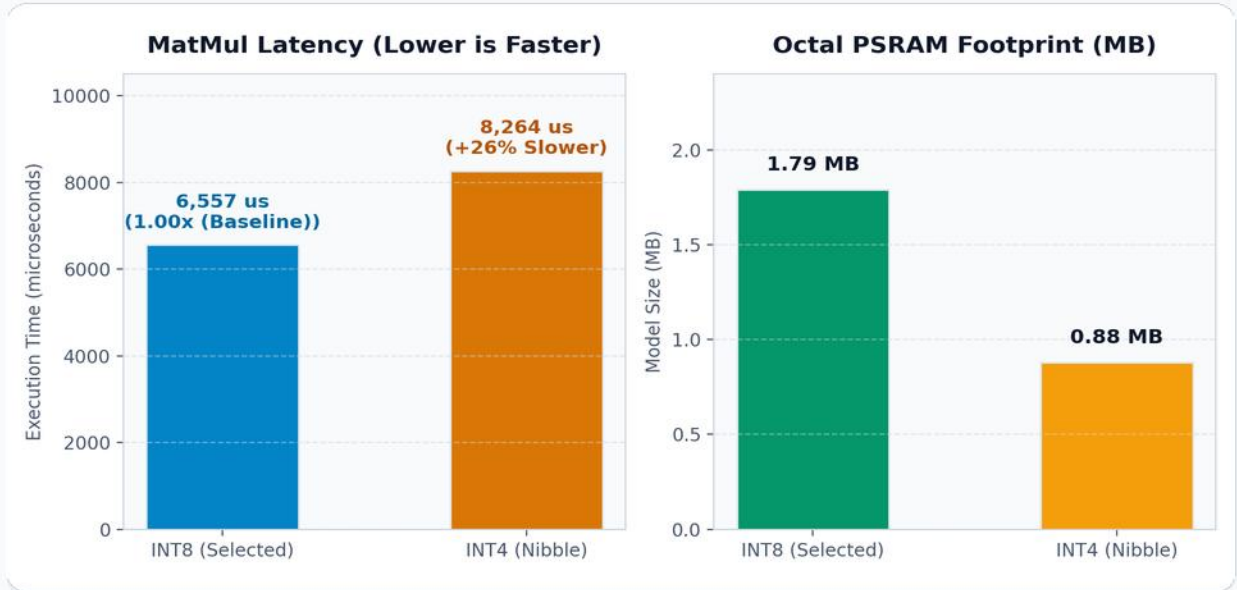
05

Hardware ALU Intercept

Math questions ('100 x 7') evaluated on hardware ALU in <0.1 ms.

The INT8 vs INT4 Paradox on MCU

On-chip MatMul profiling on ESP32-S3 @ 240MHz (147,456 weights):



KEY HARDWARE FINDING

INT8 is 26% faster than INT4 on Xtensa LX7

- Without a dedicated NPU, INT4 forces the CPU to unpack each nibble using bit-masking (& 0x0F) and bit-shifting (>> 4).
- This ALU unpacking overhead wastes more CPU cycles than the memory bandwidth savings it provides.
- Byte-aligned INT8 remains the optimal sweet spot for microcontrollers!

Real-Time UART Generation (~12-13 tok/s)

```
serial_chat.py - 115200 baud

import threading
8

Problems Output Debug Console Terminal Ports ESP-IDF
(base) aiot@aiot-Legion-T7-DLCB-03:~/Projects/KIBO-MICROLMS ./chat kibo_esp32_live.sh
kibo-client: connected to /dev/ttyUSB0 @ 115200 baud (Ctrl+C to exit)

ESP-ROM:esp32s3-20210327
Build:Mar 27 2021
rst:0x1 (POWERON),boot:0x28 (SPI_FAST_FLASH_BOOT)
SPIWP:0xee
mode:DIO, clock div:1
load:0x3fce2820,len:0xfbf0
load:0x403c8700,len:0xaf4
load:0x403cb700,len:0x2e24
entry 0x403c8898

kibo-mcu [1.84M params | int8 | xtensa-lx7 @ 240MHz]
heap: 313524 bytes | psram: 8384788 bytes
[kibo] working buffers allocated (302116 bytes free heap)
[kibo] psram buffer loaded at 0x3C2D4EF4 (1.79 MB)
[kibo] tensor layout: vocab=91 embd=192 layers=4 tensors=53
[kibo] int8 inference engine initialized
ready. enter prompt (e.g. 'hello kibo', 'what is 25 times 4')

User: hello

User: hello
Kibo: [HAPPY] Hello there! Great to see you! How can Kibo help you today?

< [63 tokens generated in 5.17 s | Speed: 12.2 tok/s]

User: how are you?

User: how are you?
Kibo: [HAPPY] I am feeling energetic and super ready to help you!

< [55 tokens generated in 4.53 s | Speed: 12.2 tok/s]

User: what is 100 times 7?

User: what is 100 times 7?
[Tool CALC: 100 * 7 = 700]
Kibo: [HAPPY] 100 times 7 is 700! Kibo is ready to help you!
```

12.2 tok/s

Real-Time Speed

Physical chip verified

1.79 MB

PSRAM Footprint

8MB Octal PSRAM

 $< 0.1 \text{ ms}$

ALU Math Tool

Zero hallucinations

Validated on 3 Form Factors



ESP32-S3 DevKit N16R8

16MB Flash | 8MB Octal PSRAM

✓ 100% Tested & Verified on Hardware



Arduino Nano ESP32

16MB Flash | 8MB Octal PSRAM

✓ 100% Tested & Verified on Hardware



Seeed Studio XIAO S3

8MB Flash | 8MB Octal PSRAM

✓ 100% Tested & Verified on Hardware

100% Open Source on GitHub (MIT License)

github.com/fitranurmayadi/esp32-microlm

Clone, train your custom dataset, and flash to your ESP32!